

Murilo Tomaz Gonzaga
Prof. Douglas Rodrigues
Métodos Numéricos Computacionais
19 de junho de 2025

Relatório do Trabalho prático 5 - Ajuste de Curvas

Implementação:

Foi utilizada a linguagem de programação Python para implementar o programa. Dentre as bibliotecas utilizadas, destaco:

- matplotlib para geração de gráficos;
- numpy para cálculo de matrizes e vetores;
- math para funções matemáticas, como o logaritmo natural;
- tabulate para a exibição tabular dos dados.

A ideia central do projeto foi a implementação de métodos de ajuste de curvas para representar o crescimento populacional ao longo do tempo, utilizando as técnicas aprendidas em aula. Foram implementados os seguintes ajustes:

- Ajuste Linear (1º grau)
- Ajuste Polinomial de 2º grau
- Ajuste Polinomial de 3º grau
- Ajuste Polinomial de 4º grau
- Ajuste Exponencial

Todas essas implementações são gerenciadas por uma interface em modo texto (CMD), com menu interativo para escolha das operações a serem realizadas pelo usuário.

Imports e Variáveis Globais:

```
import os
import numpy as np
import matplotlib.pyplot as plt
from math import sqrt, log, e
from tabulate import tabulate
```

Foram importadas bibliotecas que otimizam a execução do código e tornam a implementação mais enxuta. Os dados utilizados no problema (tempo e população) foram armazenados em arrays do tipo numpy, o que permitiu operações vetoriais rápidas e eficientes.

Funções Utilitárias:

O código foi segmentado em funções auxiliares que realizam operações matemáticas, como cálculo de somas, potências de x , produtos mistos (ex: $x^2 * y$), além do cálculo de resíduos e do coeficiente de determinação (R^2).

Apesar do numpy já possuir diversas funções que poderiam automatizar parte dessas operações, optei por criar funções específicas para facilitar o entendimento do processo de ajuste.

```
def somar_x(x):
    """Calcula a soma dos valores do vetor x"""
    return sum(x)

def somar_y(y):
    """Calcula a soma dos valores do vetor y"""
    return sum(y)

def somar_lny(y):
    """Calcula o logaritmo natural de cada elemento de y e retorna a soma e os valores"""
    ln_y = [log(yi) for yi in y]
    soma_ln_y = sum(ln_y)
    return soma_ln_y, ln_y

def somar_x_quadrado(x):
    """Calcula a soma dos quadrados dos elementos de x"""
    return sum([i**2 for i in x])

def somar_x_cubo(x):
    """Calcula a soma dos cubos dos elementos de x"""
    return sum([i**3 for i in x])

def somar_x_quarta(x):
    """Calcula a soma das quartas potências dos elementos de x"""
    return sum([i**4 for i in x])

def somar_x_quinta(x):
    """Calcula a soma das quintas potências dos elementos de x"""
    return sum([i**5 for i in x])

def somar_x_sexta(x):
    """Calcula a soma das sextas potências dos elementos de x"""
    return sum([i**6 for i in x])

def somar_x_setima(x):
    """Calcula a soma das sétimas potências dos elementos de x"""
    return sum([i**7 for i in x])

def somar_x_oitava(x):
    """Calcula a soma das oitavas potências dos elementos de x"""
    return sum([i**8 for i in x])

def somar_xy(x, y):
    """Calcula a soma dos produtos x*y para cada par de elementos"""
    return sum([xi * yi for xi, yi in zip(x, y)])

def somar_xQy(x, y):
    """Calcula a soma dos produtos x^2*y para cada par de elementos"""
    return sum([xi**2 * yi for xi, yi in zip(x, y)])

def somar_xCy(x, y):
    """Calcula a soma dos produtos x^3*y para cada par de elementos"""
    return sum([xi**3 * yi for xi, yi in zip(x, y)])

def somar_xQCy(x, y):
    """Calcula a soma dos produtos x^4*y para cada par de elementos"""
    return sum([xi**4 * yi for xi, yi in zip(x, y)])
```

```

def SQR(y, yy):
    """Calcula a Soma dos Quadrados dos Resíduos"""
    return sum([(yi - yyi)**2 for yi, yyi in zip(y, yy)])

def SQT(y, yy):
    """Calcula a Soma dos Quadrados Totais"""
    n = len(yy)
    soma = sum(yy)
    media = soma / n
    return sum([(yi - media)**2 for yi in y])

def RQ(sqt, sqr):
    """Calcula o Coeficiente de Determinação R²"""
    return 1 - (sqr/sqt)

def erro_medio(sqr, n):
    """Calcula o Erro Quadrático Médio"""
    return sqrt(sqr/n)

```

Métodos Numéricos de Ajuste:

```

def ajuste_linear(x, y):
    """Realiza ajuste linear (1º grau) y = ax + b"""
    sumX = somar_x(x)
    sumY = somar_y(y)
    sumXQ = somar_x_quadrado(x)
    sumXY = somar_xy(x, y)

    total = len(x)

    A = np.array([[total, sumX],
                  [sumX, sumXQ]])
    vb = np.array([sumY, sumXY])

    s = np.linalg.solve(A, vb)

    b = s[0]
    a = s[1]

    return a, b

```

```
def ajuste_polinomial_2(x, y):
    """Realiza ajuste polinomial de 2° grau  $y = ax^2 + bx + c$ """
    sumX = somar_x(x)
    sumXQ = somar_x_quadrado(x)
    sumXC = somar_x_cubo(x)
    sumXQQ = somar_x_quarta(x)
    sumY = somar_y(y)
    sumXY = somar_xy(x, y)
    sumXQY = somar_xQy(x, y)

    total = len(x)

    A = np.array([[total, sumX, sumXQ],
                  [sumX, sumXQ, sumXC],
                  [sumXQ, sumXC, sumXQQ]])
    vb = np.array([sumY, sumXY, sumXQY])

    s = np.linalg.solve(A, vb)

    c = s[0]
    b = s[1]
    a = s[2]

    return a, b, c
```

```
def ajuste_polinomial_3(x, y):
    """Realiza ajuste polinomial de 3° grau  $y = ax^3 + bx^2 + cx + d$ """
    sumX = somar_x(x)
    sumXQ = somar_x_quadrado(x)
    sumXC = somar_x_cubo(x)
    sumXQQ = somar_x_quarta(x)
    sumXQI = somar_x_quinta(x)
    sumXS = somar_x_sexta(x)
    sumY = somar_y(y)
    sumXY = somar_xy(x, y)
    sumXQY = somar_xQy(x, y)
    sumXCY = somar_xCy(x, y)

    total = len(x)

    A = np.array([[total, sumX, sumXQ, sumXC],
                  [sumX, sumXQ, sumXC, sumXQQ],
                  [sumXQ, sumXC, sumXQQ, sumXQI],
                  [sumXC, sumXQQ, sumXQI, sumXS]])
    vb = np.array([sumY, sumXY, sumXQY, sumXCY])

    s = np.linalg.solve(A, vb)

    d = s[0]
    c = s[1]
    b = s[2]
    a = s[3]

    return a, b, c, d
```

```
def ajuste_polynomial_4(x, y):
    """Realiza ajuste polinomial de 4° grau  $y = ax^4 + bx^3 + cx^2 + dx + e$ """
    sumX = somar_x(x)
    sumXQ = somar_x_quadrado(x)
    sumXC = somar_x_cubo(x)
    sumXQQ = somar_x_quarta(x)
    sumXQI = somar_x_quinta(x)
    sumXS = somar_x_sexta(x)
    sumXST = somar_x_setima(x)
    sumXO = somar_x_oitava(x)
    sumY = somar_y(y)
    sumXY = somar_xy(x, y)
    sumXQY = somar_xQy(x, y)
    sumXCy = somar_xCy(x, y)
    sumXQQY = somar_xQQy(x, y)

    total = len(x)

    A = np.array([[total, sumX, sumXQ, sumXC, sumXQQ],
                  [sumX, sumXQ, sumXC, sumXQQ, sumXQI],
                  [sumXQ, sumXC, sumXQQ, sumXQI, sumXS],
                  [sumXC, sumXQQ, sumXQI, sumXS, sumXST],
                  [sumXQQ, sumXQI, sumXS, sumXST, sumXO]])
    vb = np.array([sumY, sumXY, sumXQY, sumXCy, sumXQQY])

    s = np.linalg.solve(A, vb)

    ee = s[0]
    d = s[1]
    c = s[2]
    b = s[3]
    a = s[4]

    return a, b, c, d, ee
```

```
def ajuste_exponencial(x, y):
    """Realiza ajuste exponencial  $y = ae^{(bx)}$ """
    sumX = somar_x(x)
    sumlnY, lny = somar_lny(y)
    sumXQ = somar_x_quadrado(x)
    sumXlnY = somar_xy(x, lny)

    total = len(x)

    A = np.array([[sumXQ, sumX],
                  [sumX, total]])
    vb = np.array([sumXlnY, sumlnY])

    s = np.linalg.solve(A, vb)

    b = s[0]
    a = s[1]

    a = e ** a # Transforma de ln(a) para a

    return a, b
```

Nesta parte do código foram implementadas funções específicas para cada tipo de ajuste:

- Ajuste Linear: baseado no método dos mínimos quadrados.
- Ajustes Polinomiais: construção de sistemas lineares para cada grau do polinômio, resolvidos com `np.linalg.solve`.
- Ajuste Exponencial: utilizando a transformação logarítmica para linearizar o modelo exponencial, e posterior cálculo dos coeficientes.

As equações geradas são posteriormente utilizadas para a construção dos gráficos e para a comparação dos modelos.

Função de Cálculo de Erros:

```
def calcular_erros(x, y):
    """Calcula e compara os erros medios de todos os ajustes"""
    n = len(x)
    a1, b1 = ajuste_linear(x, y)
    a2, b2, c2 = ajuste_polinomial_2(x, y)
    a3, b3, c3, d3 = ajuste_polinomial_3(x, y)
    a4, b4, c4, d4, ee4 = ajuste_polinomial_4(x, y)
    ae, be = ajuste_exponencial(x, y)

    y1 = a1 * x + b1
    y2 = a2 * x**2 + b2 * x + c2
    y3 = a3 * x**3 + b3 * x**2 + c3 * x + d3
    y4 = a4 * x**4 + b4 * x**3 + c4 * x**2 + d4 * x + ee4
    ye = ae * np.exp(be * x)

    sqr1 = SQR(y, y1)
    sqr2 = SQR(y, y2)
    sqr3 = SQR(y, y3)
    sqr4 = SQR(y, y4)
    sqr5 = SQR(y, ye)

    e1 = erro_medio(sqr1, n)
    e2 = erro_medio(sqr2, n)
    e3 = erro_medio(sqr3, n)
    e4 = erro_medio(sqr4, n)
    e5 = erro_medio(sqr5, n)

    print(f"Ajuste Linear: erro médio = {e1:.4f}")
    print(f"Polinomial 2º Grau: erro médio = {e2:.4f}")
    print(f"Polinomial 3º Grau: erro médio = {e3:.4f}")
    print(f"Polinomial 4º Grau: erro médio = {e4:.4f}")
    print(f"Ajuste Exponencial: erro médio = {e5:.4f}")

    modelos = ['Linear', 'Polinomial 2º grau', 'Polinomial 3º grau', 'Polinomial 4º grau', 'Exponencial']
    valores_rq = [e1, e2, e3, e4, e5]

    plt.figure(figsize=(8,5))
    bars = plt.bar(modelos, valores_rq, color='skyblue')
    plt.ylim(0, max(valores_rq)*1.1) # Ajuste dinâmico do limite y
    plt.ylabel('Erro Quadrático Médio')
    plt.title('Comparação dos Erros Médios dos Ajustes')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    # Mostrar os valores em cima das barras
    for bar, valor in zip(bars, valores_rq):
        altura = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, altura + 0.02, f'{valor:.4f}',
                 ha='center', va='bottom', fontsize=9)

    plt.tight_layout()
    plt.savefig('erros_medio.png')
    plt.close()

    input("Pressione qualquer tecla para continuar...")
```

A função `calcular_erros()` avalia e compara o Erro Quadrático Médio (EQM) de cada ajuste, permitindo analisar a precisão de cada modelo.

Um gráfico de barras é gerado para facilitar a comparação visual entre os métodos.

Função de Cálculo do Coeficiente de Determinação:

```
def calcular_coeficientes(x, y):
    """Calcula e compara os coeficientes de determinação R² de todos os ajustes"""
    a1, b1 = ajuste_linear(x, y)
    a2, b2, c2 = ajuste_polinomial_2(x, y)
    a3, b3, c3, d3 = ajuste_polinomial_3(x, y)
    a4, b4, c4, d4, ee4 = ajuste_polinomial_4(x, y)
    ae, be = ajuste_exponencial(x, y)

    y1 = a1 * x + b1
    y2 = a2 * x**2 + b2 * x + c2
    y3 = a3 * x**3 + b3 * x**2 + c3 * x + d3
    y4 = a4 * x**4 + b4 * x**3 + c4 * x**2 + d4 * x + ee4
    ye = ae * np.exp(be * x)

    sqr1 = SQR(y, y1)
    sqr2 = SQR(y, y2)
    sqr3 = SQR(y, y3)
    sqr4 = SQR(y, y4)
    sqr5 = SQR(y, ye)

    sqt = SQT(y, y)

    rq1 = RQ(sqt, sqr1)
    rq2 = RQ(sqt, sqr2)
    rq3 = RQ(sqt, sqr3)
    rq4 = RQ(sqt, sqr4)
    rqe = RQ(sqt, sqr5)

    print(f"Ajuste Linear: R² = {rq1:.4f}")
    print(f"Polinomial 2º Grau: R² = {rq2:.4f}")
    print(f"Polinomial 3º Grau: R² = {rq3:.4f}")
    print(f"Polinomial 4º Grau: R² = {rq4:.4f}")
    print(f"Ajuste Exponencial: R² = {rqe:.4f}")

    modelos = ['Linear', 'Polinomial 2º grau', 'Polinomial 3º grau', 'Polinomial 4º grau', 'Exponencial']
    valores_rq = [rq1, rq2, rq3, rq4, rqe]

    plt.figure(figsize=(8,5))
    bars = plt.bar(modelos, valores_rq, color='skyblue')
    plt.ylim(0, 1)
    plt.ylabel('Coeficiente de Determinação (R²)')
    plt.title('Comparação dos R² dos Ajustes')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    # Mostrar os valores em cima das barras
    for bar, valor in zip(bars, valores_rq):
        altura = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, altura + 0.02, f'{valor:.4f}',
                 ha='center', va='bottom', fontsize=9)

    plt.tight_layout()
    plt.savefig('coeficientes_determinacao.png')
    plt.close()

    input("Pressione qualquer tecla para continuar...")
```

```
def calcular_ajustes(x, y):
    """Calcula todos os ajustes e gera gráficos individuais e comparativos"""
    os.system('cls')
    a1, b1 = ajuste_linear(x, y)
    a2, b2 = ajuste_polinomial_2(x, y)
    a3, b3, c3, d3 = ajuste_polinomial_3(x, y)
    a4, b4, c4, d4, ee4 = ajuste_polinomial_4(x, y)
    ae, be = ajuste_exponencial(x, y)

    print("Ajuste linear: {:.4f}x + {:.4f}".format(a1, b1))
    print("Ajuste polinomial de 2º grau: {:.4f}x^2 + {:.4f}x + {:.4f}".format(a2, b2, c2))
    print("Ajuste polinomial de 3º grau: {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}".format(a3, b3, c3, d3))
    print("Ajuste polinomial de 4º grau: {:.4f}x^4 + {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}".format(a4, b4, c4, d4, ee4))
    print("Ajuste exponencial: {:.4f} * e ^ {:.4f}x".format(ae, be))

    x_smooth = np.linspace(min(x), max(x), 200)

    # Funções para plotagem
    y_linear = a1 * x_smooth + b1
    y_poly2 = a2 * x_smooth**2 + b2 * x_smooth + c2
    y_poly3 = a3 * x_smooth**3 + b3 * x_smooth**2 + c3 * x_smooth + d3
    y_poly4 = a4 * x_smooth**4 + b4 * x_smooth**3 + c4 * x_smooth**2 + d4 * x_smooth + ee4
    y_exp = ae * np.exp(be * x_smooth)

    def salvar_grafico(x, y, y_fit, titulo, filename):
        """Função auxiliar para salvar gráficos individuais"""
        plt.figure(figsize=(8, 5))
        plt.scatter(x, y, color='black', label='Dados observados')
        plt.plot(x_smooth, y_fit, label=titulo, color='red')
        plt.xlabel('Tempo (anos)')
        plt.ylabel('População (milhares)')
        plt.title(titulo)
        plt.legend()
        plt.grid(True)
        plt.savefig(filename)
        plt.close()

    # Salvar gráficos individuais
    salvar_grafico(x, y, y_linear, 'Ajuste Linear', 'ajuste_linear.png')
    salvar_grafico(x, y, y_poly2, 'Ajuste Polinomial 2º grau', 'ajuste_polinomial_2.png')
    salvar_grafico(x, y, y_poly3, 'Ajuste Polinomial 3º grau', 'ajuste_polinomial_3.png')
    salvar_grafico(x, y, y_poly4, 'Ajuste Polinomial 4º grau', 'ajuste_polinomial_4.png')
    salvar_grafico(x, y, y_exp, 'Ajuste Exponencial', 'ajuste_exponencial.png')

    # Gráfico geral com todos os ajustes
    plt.figure(figsize=(10, 6))
    plt.scatter(x, y, color='black', label='Dados observados')
    plt.plot(x_smooth, y_linear, label='Ajuste Linear')
    plt.plot(x_smooth, y_poly2, label='Polinomial 2º grau')
    plt.plot(x_smooth, y_poly3, label='Polinomial 3º grau')
    plt.plot(x_smooth, y_poly4, label='Polinomial 4º grau')
    plt.plot(x_smooth, y_exp, label='Ajuste Exponencial')
    plt.xlabel('Tempo (anos)')
    plt.ylabel('População (milhares)')
    plt.title('Ajustes das funções aos dados observados')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('ajustes_geral.png')
    plt.close()
```

A função `calcular_coeficientes()` calcula e compara os valores de R^2 de todos os ajustes, permitindo avaliar o quão bem cada curva se ajusta aos dados observados.

Também é gerado um gráfico de barras para facilitar a análise dos resultados.

Função de Geração de Gráficos:

A função `calcular_ajustes()` é responsável por:

- Exibir a equação de cada ajuste;
- Gerar gráficos individuais para cada tipo de ajuste;
- Gerar um gráfico comparativo com todos os ajustes sobrepostos aos dados observados.

Essa abordagem visual ajuda a entender o comportamento de cada modelo frente aos dados reais.

Função Main:

```
def main():
    """Função principal que gerencia o menu e a execução do programa"""
    os.system("cls")
    print("Problema - Modelagem de Crescimento Populacional")
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([10, 12.5, 15.6, 19.5, 24.4, 30.5, 38.1, 47.6, 59.5, 74.4])

    # Exibe os dados iniciais
    tabela = list(zip(x, y))
    cabecalhos = ["Tempo (anos)", "População (milhares)"]

    # Menu interativo
    while True:
        os.system("cls")
        print(tabulate(tabela, headers=cabecalhos, tablefmt="grid"))
        print("\nMenu de Análise de Ajustes")
        print("1. Calcular ajustes")
        print("2. Comparar coeficientes de determinação (R²)")
        print("3. Comparar erros quadraticos médios")
        print("4. Sair")

        try:
            opcao = int(input("Escolha uma opção: "))
        except ValueError:
            print("Entrada inválida. Digite um número entre 1 e 4.")
            input("Pressione Enter para continuar...")
            continue

        if opcao == 1:
            calcular_ajustes(x, y)
        elif opcao == 2:
            calcular_coeficientes(x, y)
        elif opcao == 3:
            calcular_erros(x, y)
        elif opcao == 4:
            print("Encerrando o programa...")
            break
        else:
            print("Opção inválida! Digite um número entre 1 e 4.")
            input("Pressione Enter para continuar...")

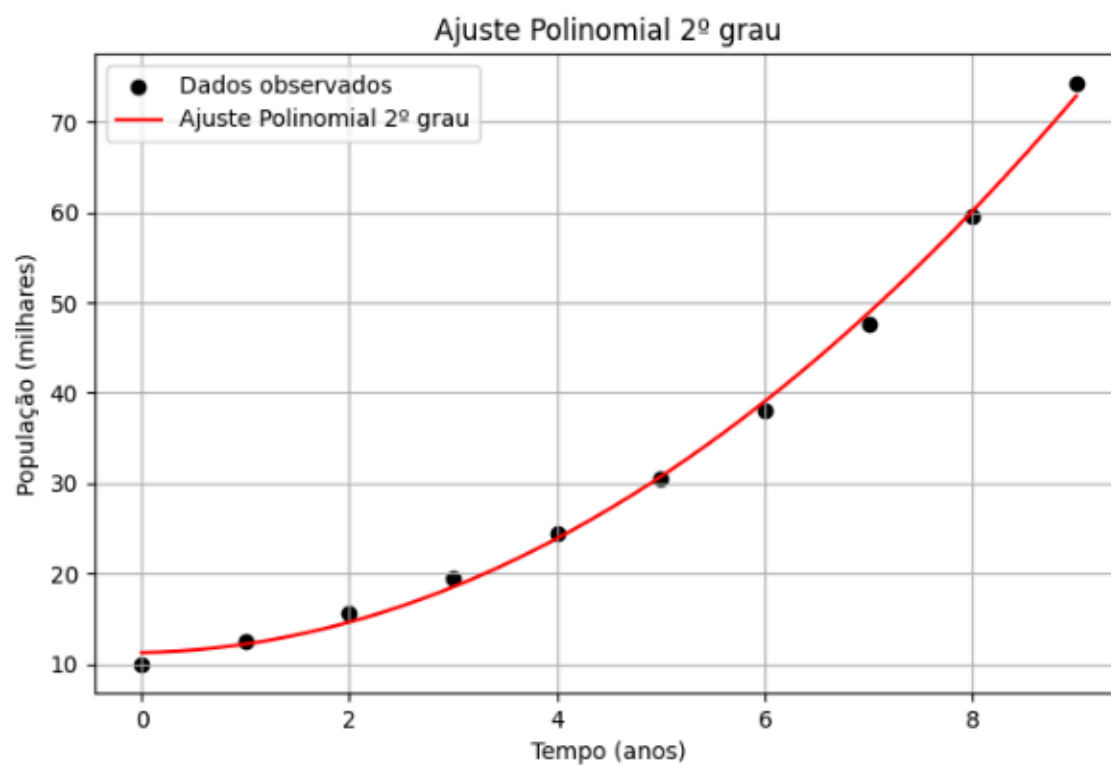
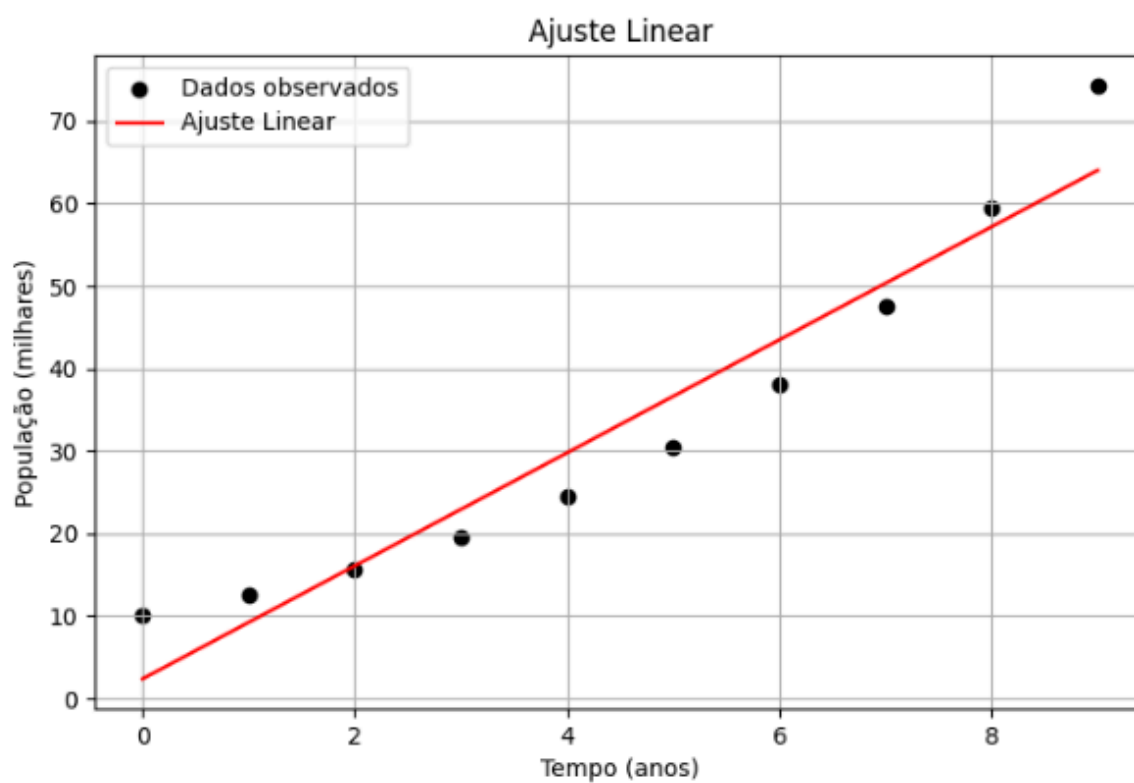
if __name__ == "__main__":
    main()
```

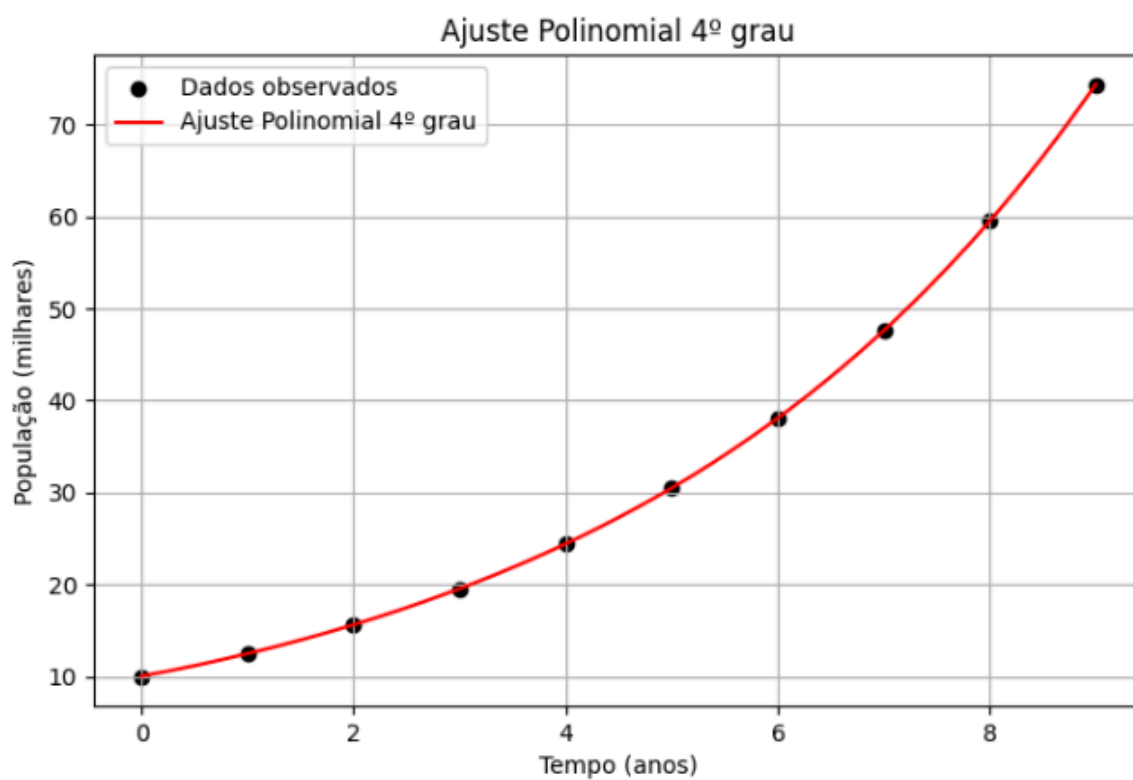
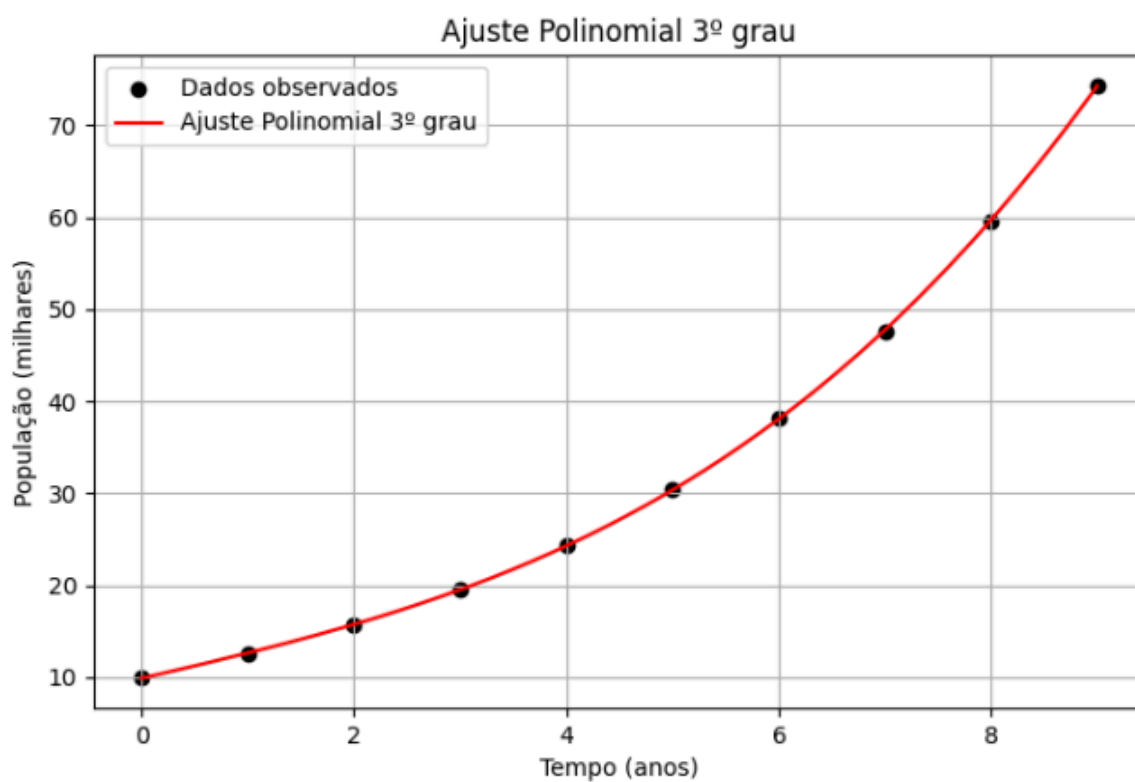
A função main() apresenta uma interface amigável para o usuário, permitindo que ele escolha, por meio do menu interativo:

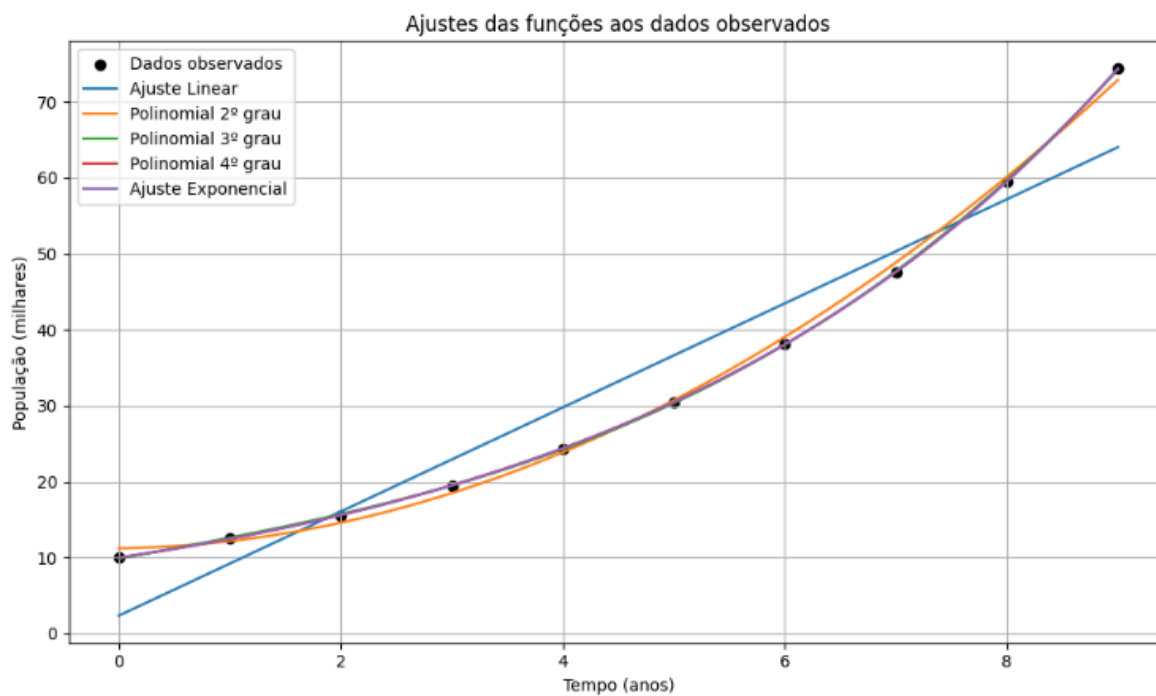
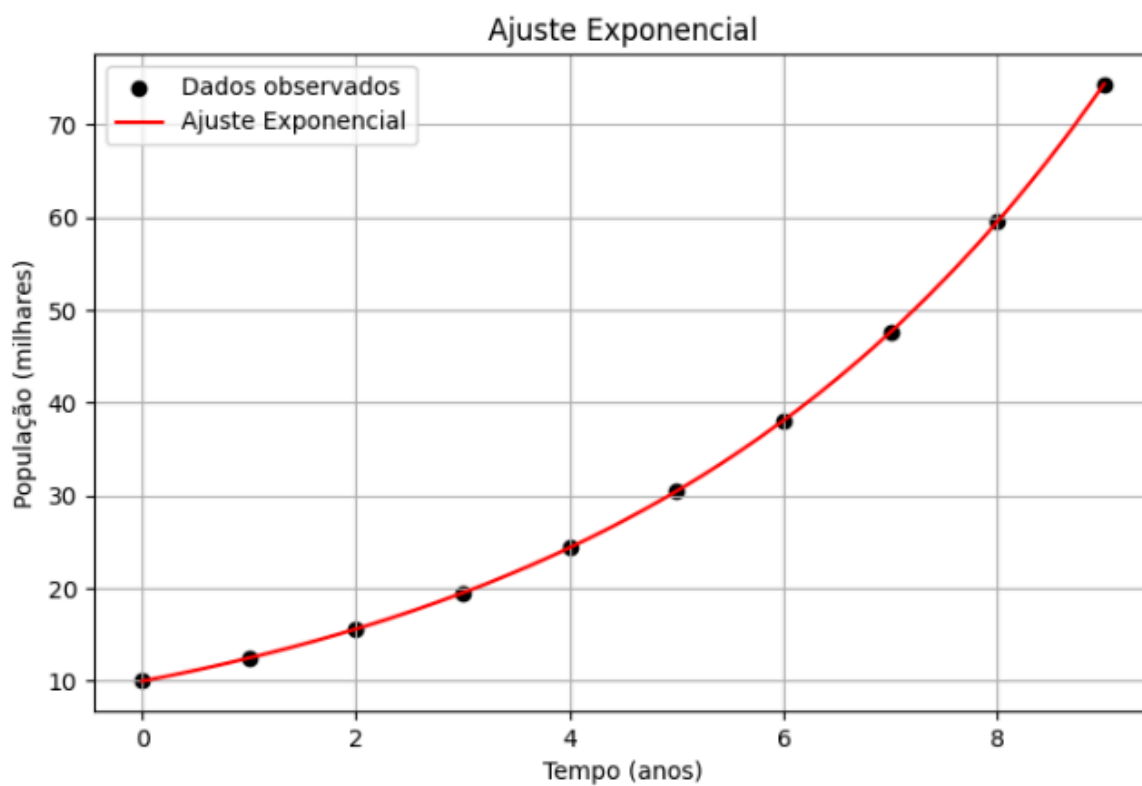
1. Calcular os ajustes e visualizar os gráficos;
2. Comparar os coeficientes de determinação (R^2);
3. Comparar os erros quadráticos médios;
4. Encerrar o programa.

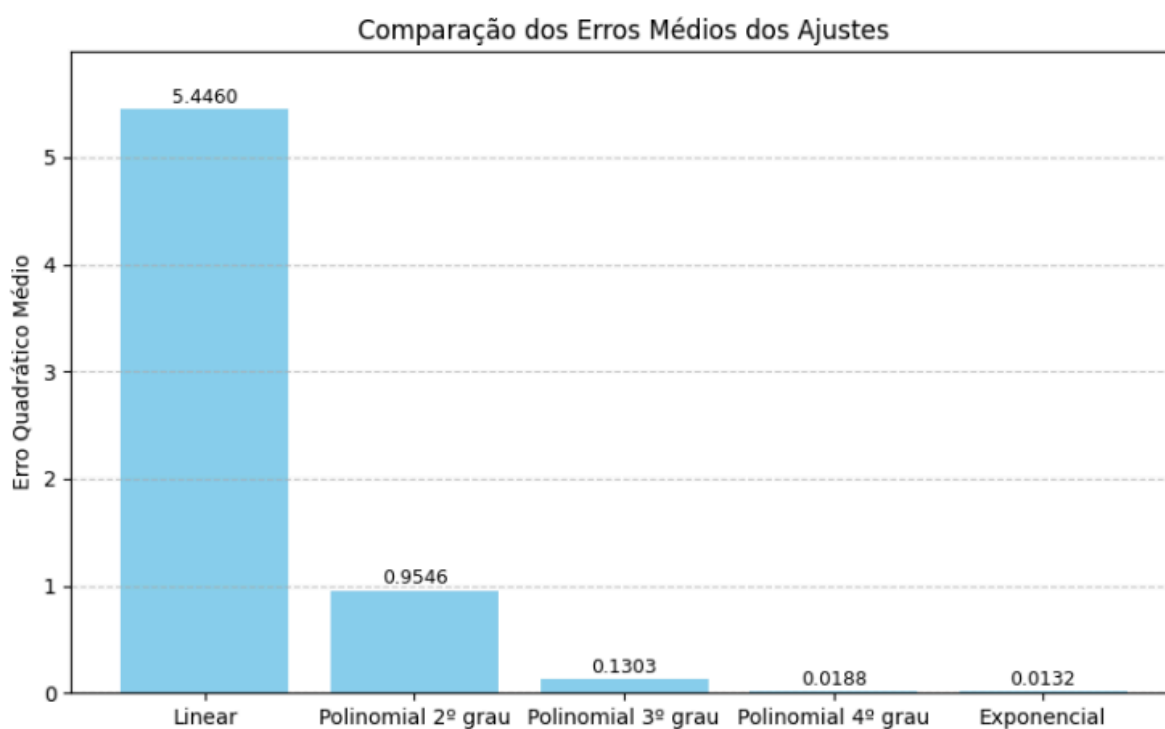
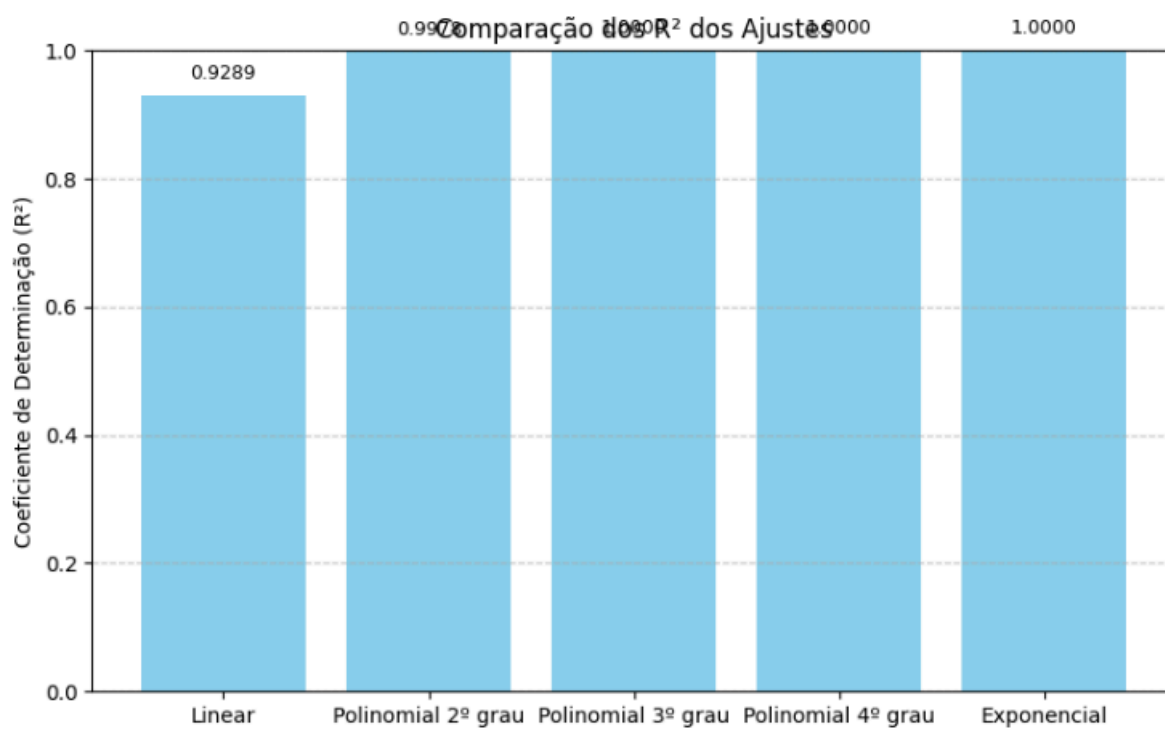
A interface foi implementada com try-except, garantindo robustez contra entradas inválidas.

Análises Gráficas:









Conclusão:

Com este trabalho, ficou evidente a importância dos métodos de ajuste de curvas para a modelagem de fenômenos como o crescimento populacional.

A implementação foi um exercício valioso para aprofundar o entendimento dos conceitos teóricos de regressão e modelagem matemática, bem como para reforçar o uso de ferramentas computacionais para análise de dados.

Os resultados obtidos demonstraram que:

- Os modelos polinomiais de grau mais alto apresentam maior aderência aos dados (R^2 mais próximo de 1), mas podem sofrer com overfitting;
- O modelo exponencial se mostrou bastante adequado para o tipo de crescimento populacional analisado, com bom desempenho em termos de erro médio e R^2 ;
- A interface gráfica gerada com matplotlib facilita consideravelmente a interpretação dos resultados.

A proposta de utilizar diferentes tipos de ajustes e compará-los entre si proporcionou uma compreensão mais profunda das vantagens e limitações de cada modelo. Além disso, a experiência reforçou a importância da programação como ferramenta fundamental no campo da modelagem matemática e dos métodos numéricos.